ExamWise: AI-Driven Educational Analytics and Assessment Design Assistant

Final Project Report — Milestones 1 & 2

Agentic AI Systems for Education

Team EPOCH PIONEERS

Department of Computer Science and Engineering

April 19, 2026

Abstract

Designing balanced and pedagogically sound examinations is a persistent challenge in education. Educators typically discover that an exam was poorly calibrated only *after* students have already taken it, when it is too late to adjust. This project presents **ExamWise**, an end-to-end AI-driven educational analytics platform that addresses this problem in two phases. In **Milestone 1**, we build a machine learning and natural language processing (NLP) pipeline that ingests question–answer data from Stack Overflow, engineers engagement-based difficulty features, and trains a Logistic Regression classifier to predict question difficulty as *Easy*, *Medium*, or *Hard*. The model achieves 65.71% overall accuracy on the test set. In **Milestone 2**, we extend the system with an **Agentic AI Assessment Design Assistant** powered by the Google Gemini large language model (LLM). The agent performs stateful analysis of assessment data, retrieves pedagogical best practices through a synthetic Retrieval-Augmented Generation (RAG) mechanism, generates structured six-section assessment improvement reports, and handles incomplete or noisy data gracefully. As an extension, we implement **Automated Question Generation**, allowing educators to instantly create new exam questions for identified weak topics. The entire application is built using Streamlit and is publicly deployable on free-tier hosting platforms.

Contents

1 Introduction **4**
- 1.1 Motivation and Problem Statement 4
- 1.2 Scope of This Report 4
- 1.3 Constraints and Requirements 4

2 Literature Review **4**
- 2.1 Automated Question Difficulty Estimation 4

2.1.1 Item Response Theory (IRT) 5
2.1.2 Text-Based Approaches 5
2.1.3 Community Q&A Platforms as Proxies 5
2.2 Large Language Models in Education 5
2.3 Agentic AI Systems 5
2.4 Retrieval-Augmented Generation (RAG) 5

3 Dataset **6**
3.1 Data Source 6
3.2 Data Reduction Strategy 6
3.3 Why Stack Overflow Data? 6

4 Methodology: Milestone 1 — ML-Based Analytics **7**
4.1 System Architecture Overview 7
4.2 Feature Engineering 7
4.2.1 Step 1: Answer Aggregation 7
4.2.2 Step 2: Difficulty Score Computation 8
4.2.3 Step 3: Difficulty Categorisation 8
4.3 Label Distribution 8
4.4 Text Preprocessing 8
4.5 Text Vectorisation 9
4.6 Classification Model 9
4.7 Train–Test Split 9
4.8 Model Persistence 9

5 Results: Milestone 1 **10**
5.1 Classification Performance 10
5.2 Analysis of Results 10
5.3 Visualisations 11
5.4 Hardest Questions Analysis 12

6 Methodology: Milestone 2 — Agentic AI Assistant **12**
6.1 Objectives 12
6.2 LLM Selection: Google Gemini 13
6.3 Agent Architecture 13
6.3.1 Component 1: Explicit State Management 13
6.3.2 Component 2: Synthetic Retrieval-Augmented Generation (RAG) 14
6.3.3 Component 3: Data Validation and Noisy Data Handling 14
6.4 Structured Report Generation 15
6.5 Prompt Engineering Strategies 15
6.6 Extension: Automated Question Generation 15

7 System Implementation **16**
7.1 Technology Stack 16
7.2 Project Structure 16
7.3 User Interface Design 17
7.4 Security Considerations 17

8 Results: Milestone 2 **17**
8.1 Sample Assessment Report 17
8.2 Sample Generated Question 18
8.3 Noisy Data Handling Demonstration 18

9 Discussion **18**
9.1 Strengths 18
9.2 Limitations 18

10 Future Work **19**

11 Conclusion **20**

A Key Code Listings **22**
A.1 Difficulty Score Computation 22
A.2 Agent Prompt Construction 22

B Team Contributions **23**

1 Introduction

1.1 Motivation and Problem Statement

In conventional educational assessment workflows, question difficulty is understood only *retrospectively*—after students take the exam and aggregate scores are computed [Lord, 1980]. This reactive paradigm creates two significant problems:

1. **Poorly balanced exams**: Without prior difficulty estimation, exams may be disproportionately easy or hard, failing to differentiate student abilities effectively.
2. **Missed learning gaps**: Patterns of student struggle on specific topics are only visible after grading, leaving no opportunity for proactive pedagogical intervention.

ExamWise addresses both problems by building a predictive and generative AI system that:

- Predicts question difficulty from text *before* the exam is administered.
- Analyzes difficulty distributions and learning gaps *after* an exam to guide future assessment design.
- Generates structured, actionable recommendations grounded in established pedagogical principles.
- Automatically creates new questions for topics identified as weak areas.

1.2 Scope of This Report

This report documents the complete development of the ExamWise system across two milestones:

- **Milestone 1 (Mid-Semester):** ML-based exam question analytics—data pipeline, feature engineering, supervised classification, model evaluation, and basic Streamlit UI.
- **Milestone 2 (End-Semester):** Agentic AI Assessment Design Assistant—LLM integration, synthetic RAG, structured report generation, noisy data handling, and automated question generation.

1.3 Constraints and Requirements

The project was developed under the following constraints:

- Team size: 3–4 students.
- **No paid APIs permitted**—only free-tier APIs or open-source models.
- A user interface is mandatory.
- The final application must be publicly hosted (not localhost-only).

2 Literature Review

2.1 Automated Question Difficulty Estimation

The estimation of question difficulty from textual and contextual features has been explored through several complementary approaches in educational data mining and psychometrics.

2.1.1 Item Response Theory (IRT)

Classical psychometric models, particularly Item Response Theory [Hambleton et al., 1991], estimate item difficulty as a latent parameter from observed student response patterns. The standard three-parameter logistic (3PL) model defines the probability of a correct response as:

$$P(\text{correct} \mid \theta) = c + \frac{1-c}{1+e^{-a(\theta-b)}} \tag{1}$$

where θ is the student ability, b is the item difficulty, a is the discrimination parameter, and c is the guessing parameter. While IRT provides rigorous statistical foundations, it *requires observed response data* and cannot predict difficulty for new, untested items.

2.1.2 Text-Based Approaches

Recent work uses NLP features—readability scores, syntactic complexity, word frequency distributions—to predict difficulty from question text alone [Ha et al., 2019]. Common text representations include TF-IDF vectors and pre-trained word embeddings. These approaches enable *a priori* difficulty estimation without requiring student response data.

2.1.3 Community Q&A Platforms as Proxies

Stack Overflow question–answer data has been used as a proxy for educational content difficulty [Anderson et al., 2012]. Metrics such as answer scores, response rates, and time-to-first-answer serve as engagement-based difficulty indicators. Our approach follows this paradigm, using community engagement to derive difficulty labels.

2.2 Large Language Models in Education

The emergence of large language models (LLMs) has created new possibilities for educational technology. LLMs can generate human-quality text, reason about complex topics, and follow structured output formats [Brown et al., 2020]. However, they also carry risks of generating plausible but incorrect information ("hallucination"), which is particularly dangerous in educational contexts where accuracy is paramount.

2.3 Agentic AI Systems

Agentic AI refers to systems where an AI model operates with explicit state management, goal-directed reasoning, and the ability to use tools or retrieve information [Yao et al., 2023]. Unlike simple prompt-response interactions, agentic workflows maintain conversation state across multiple reasoning steps and can incorporate external knowledge bases. Our Assessment Assistant implements this pattern.

2.4 Retrieval-Augmented Generation (RAG)

RAG systems augment LLM generation with retrieved context from external knowledge bases, reducing hallucination and grounding responses in verified information [Lewis et al., 2020]. While full RAG systems use vector stores (e.g., FAISS, Chroma) for semantic retrieval, our system implements a *synthetic RAG* approach where curated pedagogical guidelines are injected directly into the prompt context.

3 Dataset

3.1 Data Source

We use the **StackSample** dataset from Kaggle [Stack Overflow, 2016], which contains a 10% random sample of all Stack Overflow questions and answers. The dataset comprises three CSV files:

Table 1: Raw dataset files and their contents

File	Key Columns	Original Rows	Description
`Questions.csv`	Id, Score, Title, Body	∼1.26M	Question text and metadata
`Answers.csv`	Id, ParentId, Score	∼1.39M	Answer text and scores
`Tags.csv`	Id, Tag	∼3.75M	Tags per question

3.2 Data Reduction Strategy

Due to memory and computational constraints, the full dataset cannot be processed on standard hardware. We implemented a **referentially-consistent reduction strategy** in `scripts/data_reduction.py` that preserves data integrity:

Algorithm 1 Referentially-Consistent Data Reduction

Require: Raw CSV files, reduction limit $L = 15{,}000$
1: Read first L rows from `Questions.csv`
2: Extract question ID set: $\mathcal{Q} = \{q_1, q_2, \ldots, q_L\}$
3: **Stream** `Answers.csv` in chunks of 50,000 rows
4: **for** each chunk **do**
5: Filter rows where `ParentId` $\in \mathcal{Q}$
6: Append filtered rows to answer buffer
7: **if** buffer size $\geq L$ **then**
8: **break**
9: **end if**
10: **end for**
11: Truncate answer buffer to L rows
12: Repeat streaming process for `Tags.csv`, filtering on `Id` $\in \mathcal{Q}$
13: Save reduced files to `data/reduced/`

The chunked reading approach (line 3) ensures that even the full 1.39M-row `Answers.csv` file can be processed without loading it entirely into memory.

3.3 Why Stack Overflow Data?

Stack Overflow questions serve as a reasonable proxy for exam questions because:

- Questions are technical and assessment-like in nature (problem-solving focus).
- Community voting (scores) provides a crowd-sourced quality/difficulty signal.
- The variety of programming topics mirrors the breadth of a computer science curriculum.
- The dataset is freely available, satisfying the no-paid-API constraint.

4 Methodology: Milestone 1 — ML-Based Analytics

4.1 System Architecture Overview

Figure 1 illustrates the end-to-end pipeline for Milestone 1.

Milestone 1 Data Flow

`Raw CSVs (Questions, Answers, Tags)`

↓ *Data Reduction (15,000 rows)*

`Reduced CSVs`

↓ *Feature Engineering*

`Processed Dataset with Difficulty Labels`

↓ *Text Preprocessing + Vectorisation*

`Bag-of-Words Feature Matrix`

↓ *Logistic Regression Training (70/30 Split)*

`Trained Model + Evaluation Metrics`

↓ *Streamlit UI*

`Interactive Dashboard with Predictions & Charts`

Figure 1: ExamWise Milestone 1 processing pipeline

4.2 Feature Engineering

The feature engineering module (`scripts/feature_engineering.py`) transforms raw question–answer data into a structured dataset with six computed features per question. The process involves three main steps:

4.2.1 Step 1: Answer Aggregation

For each question (identified by `Id`), all associated answers are grouped by `ParentId`, and the following statistics are computed:

Table 2: Engineered features per question

Feature	Computation	Intuition
`max_answer_score`	max(answer scores)	Quality of the best answer
`avg_answer_score`	mean(answer scores)	Average answer quality
`answer_score_variance`	var(answer scores)	Disagreement among answers
`answer_count`	Count of answers	Community interest level
`question_score`	Score from `Questions.csv`	Community rating of the question
`ratio`	$\frac{\text{max_answer_score}}{\text{answer_count}+1}$	Normalised answer quality

The aggregation uses `pandas.groupby()` with named aggregation functions. For questions with a single answer, the variance is set to 0 (since variance is undefined for a single observation).

4.2.2 Step 2: Difficulty Score Computation

The core difficulty metric is computed as:

$$\text{difficulty_score}_i = 1 - \frac{\text{max_answer_score}_i}{\text{max_answer_score}_i + \text{answer_count}_i + 1} \tag{2}$$

Intuition: A question with a high-scoring answer relative to its number of answers is "easy"—the community quickly converged on a good solution. Conversely, questions with low-scoring answers or very few responses are "hard"—the community struggled to provide helpful answers.

The score is then normalised to $[0, 1]$ using min–max normalisation:

$$\text{difficulty_score}_i^{\text{norm}} = \frac{\text{difficulty_score}_i - \min(\mathbf{d})}{\max(\mathbf{d}) - \min(\mathbf{d})} \tag{3}$$

where $\mathbf{d}$ is the vector of all raw difficulty scores. If all scores are identical (edge case), a fallback value of 0.5 is assigned.

4.2.3 Step 3: Difficulty Categorisation

The normalised scores are mapped to three categorical labels using fixed thresholds:

$$\text{difficulty_label}_i = \begin{cases} \textit{Easy} & \text{if difficulty_score}_i^{\text{norm}} < 0.33 \\ \textit{Medium} & \text{if } 0.33 \leq \text{difficulty_score}_i^{\text{norm}} < 0.66 \\ \textit{Hard} & \text{if difficulty_score}_i^{\text{norm}} \geq 0.66 \end{cases} \tag{4}$$

4.3 Label Distribution

The resulting class distribution is significantly imbalanced:

Table 3: Difficulty label distribution in the processed dataset

Label	Count	Percentage
Hard	11,280	75.2%
Medium	2,192	14.6%
Easy	1,528	10.2%
Total	**15,000**	**100%**

This imbalance reflects the nature of Stack Overflow: most questions are sufficiently challenging that answers do not receive overwhelmingly high scores.

4.4 Text Preprocessing

Before vectorisation, the question text (`Title` + `Body`) undergoes four preprocessing steps:

1. **HTML tag removal:** Regex pattern `<[^>]*>` strips all HTML markup from the body text.

2. **Non-alphabetic character removal:** Retains only alphabetical characters, removing code snippets, numbers, and special characters.
3. **Lowercasing:** Converts all text to lowercase for uniform representation.
4. **Stop word removal:** English stop words ("the", "is", "and", etc.) are excluded during vectorisation.

4.5 Text Vectorisation

The cleaned text is transformed into a numerical feature matrix using scikit-learn's `CountVectorizer` (Bag-of-Words model) [Pedregosa et al., 2011]. Each question is represented as a sparse vector $\mathbf{x}_i \in \mathbb{R}^{|V|}$ where $|V|$ is the vocabulary size. Each element x_{ij} represents the count of word j in question i.

4.6 Classification Model

A **Logistic Regression** classifier is trained for multi-class classification using the softmax (multinomial) formulation:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x} + b_k}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x} + b_j}} \tag{5}$$

where:

- $\mathbf{x} \in \mathbb{R}^{|V|}$ is the BoW feature vector for a question.
- $\mathbf{w}_k \in \mathbb{R}^{|V|}$ is the learned weight vector for class k.
- $b_k \in \mathbb{R}$ is the bias term for class k.
- $K = 3$ corresponds to the three classes: Easy, Medium, Hard.

The model is trained using L2 regularisation to prevent overfitting on the high-dimensional sparse feature space. The trained model and vectoriser are serialised using `joblib` [Joblib Development Team] for deployment.

4.7 Train–Test Split

The dataset is split into 70% training and 30% testing:

- Training set: 10,500 samples
- Test set: 4,500 samples

4.8 Model Persistence

Three artifacts are persisted to the `models/` directory:

- `model.joblib` (758 KB) — The trained Logistic Regression classifier.
- `vectorizer.joblib` (447 KB) — The fitted CountVectorizer with learned vocabulary.
- `metrics.txt` — The full classification report including per-class precision, recall, and F1.

5 Results: Milestone 1

5.1 Classification Performance

Table 4 presents the full classification report on the test set.

Table 4: Classification report on the test set ($n = 4{,}500$)

Class	Precision	Recall	F1-Score	Support
Easy	0.14	0.10	0.12	473
Medium	0.20	0.15	0.17	643
Hard	0.76	0.83	0.80	3,384
Overall Accuracy		**0.6571** (65.71%)		
Macro Average	0.37	0.36	0.36	4,500
Weighted Average	0.62	0.66	0.64	4,500

5.2 Analysis of Results

Key observations:

1. **Majority class dominance:** The model achieves 65.71% accuracy primarily by correctly classifying *Hard* questions (83% recall for the class comprising 75.2% of test data).

2. **Minority class struggle:** The *Easy* class achieves only 10% recall and 14% precision, meaning the model fails to identify most easy questions. This is a direct consequence of having only 473 easy questions in the test set (10.5%).

3. **Macro vs. weighted metrics:** The macro-averaged F1 of 0.36 (treating all classes equally) is significantly lower than the weighted-averaged F1 of 0.64 (accounting for class prevalence), confirming that performance is driven by the majority class.

4. **Confusion Matrix interpretation:** The confusion matrix (Figure **??**) reveals that most *Easy* and *Medium* questions are misclassified as *Hard*, consistent with the class imbalance.

5.3 Visualisations

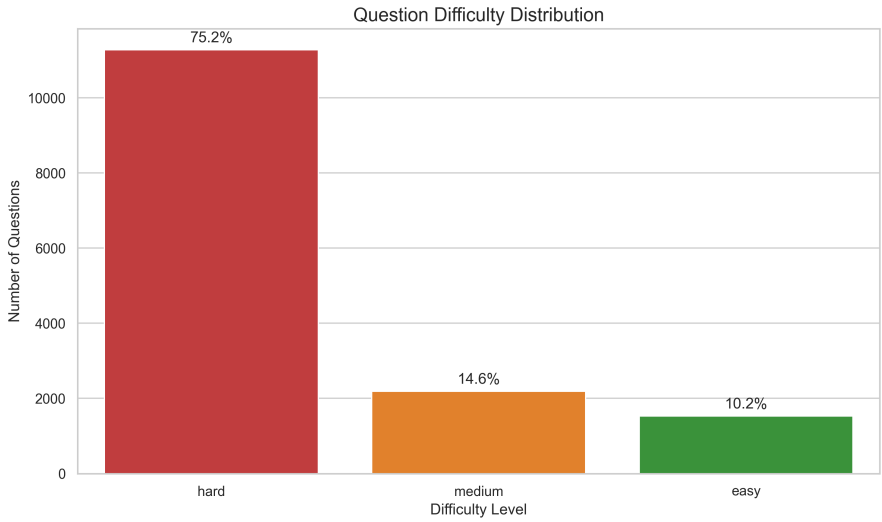

Figure 2: Distribution of difficulty labels across the processed dataset. The heavy skew towards *Hard* (75.2%) reflects the challenging nature of Stack Overflow questions and directly impacts classifier performance.

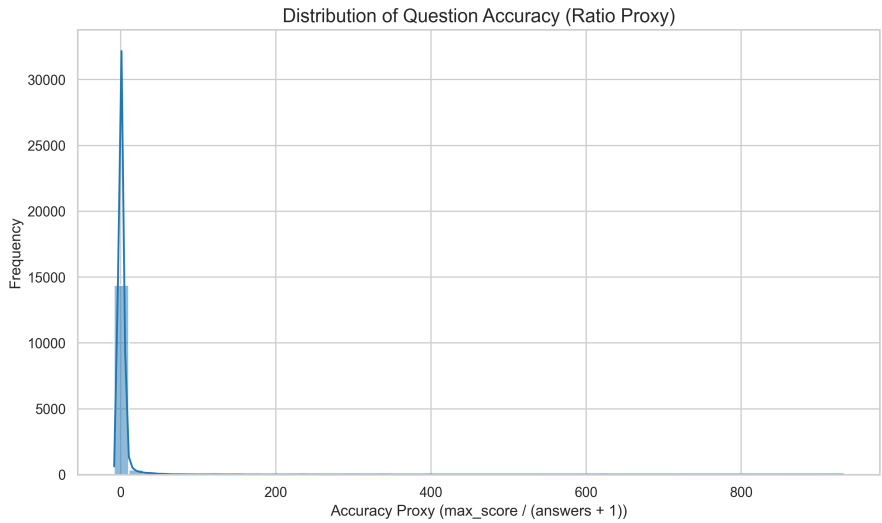

Figure 3: Distribution of the accuracy proxy metric (ratio $= \frac{\text{max_answer_score}}{\text{answer_count}+1}$). The right-skewed distribution shows that most questions have low ratios, indicating that high-quality answers are relatively rare.

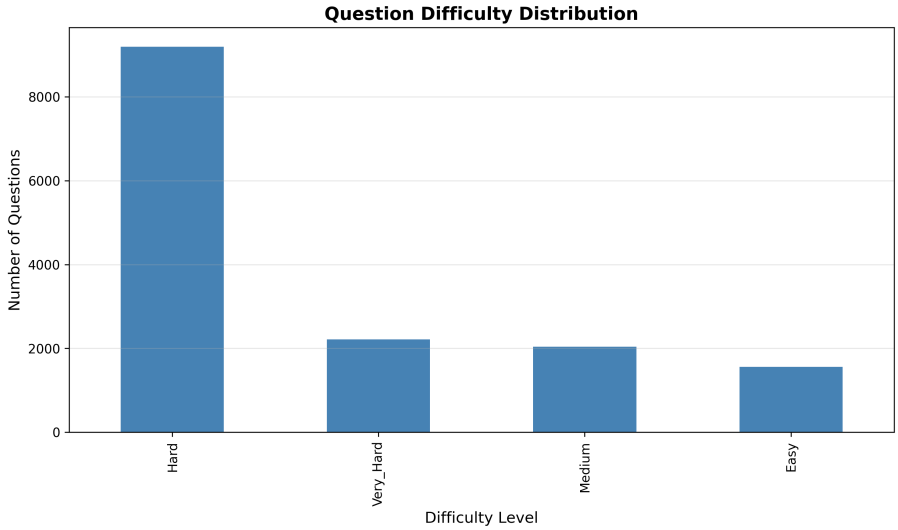

Figure 4: Bar chart of question counts per difficulty category, clearly showing the three-to-one ratio between Hard and the combined Easy+Medium classes.

5.4 Hardest Questions Analysis

Table 5 presents a sample of the most difficult questions identified by the system (those with the lowest accuracy proxy scores):

Table 5: Top 5 hardest questions identified by the system

Question ID	Title (Truncated)	Score
688245	Is there a better Python bundle for TextMate...	−17.0
183042	How can I use UUIDs in SQLAlchemy?	−15.0
688766	Getting 401 on Twitter OAuth POST requests	−1.5
284115	Cross platform hidden file detection	−1.0
336753	Django FormWizard and Admin application	−1.0

Negative scores indicate that these questions received more downvotes than upvotes, suggesting they were either poorly phrased, highly niche, or genuinely very challenging for the community.

6 Methodology: Milestone 2 — Agentic AI Assistant

6.1 Objectives

Milestone 2 extends the ML analytics pipeline into an **Agentic AI Assessment Design Assistant** that:

1. Analyzes question difficulty distribution and identifies learning gaps.
2. Retrieves pedagogical and assessment design best practices.
3. Generates structured assessment improvement recommendations.
4. Handles incomplete or noisy assessment data gracefully.
5. Generates new exam questions for weak topics (extension).

6.2 LLM Selection: Google Gemini

We selected **Google Gemini 2.5 Flash** as our LLM backend because:

- It offers a generous **free-tier API** (satisfying the no-paid-API constraint).
- It supports structured output and long context windows.
- It provides state-of-the-art reasoning capabilities comparable to GPT-4 class models.
- The Python SDK (`google-generativeai`) is simple to integrate.

6.3 Agent Architecture

The `AssessmentAgent` class (`scripts/assessment_agent.py`) implements an agentic workflow with three core components:

Milestone 2 Agent Architecture

`Processed DataFrame from M1`

↓ *1. Data Validation & Quality Check*

`Aggregated Statistics + Data Quality Notes`

↓ *2. Synthetic RAG — Inject Pedagogical Guidelines*

`Enriched Prompt Context`

↓ *3. LLM Generation (Gemini 2.5 Flash)*

`Structured 6-Section Assessment Report`

↓ *Optional: Automated Question Generation*

`New Exam Question for Weak Topic`

Figure 5: Agentic Assessment Assistant workflow

6.3.1 Component 1: Explicit State Management

The agent maintains explicit state across interactions:

```
self.state = {
    "is_configured": False,      # API key validated
    "reports_generated": 0,      # Count of reports produced
    "last_identified_gaps": []   # Topics from last analysis
}
```

Listing 1: Agent state initialisation

This state management pattern ensures that:

- The agent will not attempt LLM calls without a valid API key.
- Usage can be tracked and rate-limited if needed.
- Context from previous analyses can inform subsequent interactions.

6.3.2 Component 2: Synthetic Retrieval-Augmented Generation (RAG)

Rather than using a full vector store (FAISS, Chroma), we implement a **synthetic RAG** approach where curated pedagogical best practices are stored as a Python dictionary and injected directly into the LLM prompt. This knowledge base contains five established educational principles:

Table 6: Pedagogical knowledge base entries used for synthetic RAG

Key	Guideline
`difficulty_distribution`	A standard exam should aim for approximately 20% easy, 60% medium, and 20% hard questions to differentiate student ability while maintaining confidence.
`learning_gaps`	When students consistently score low on specific topics, it indicates a critical learning gap. Recommend targeted supplementary materials and formative assessment.
`cognitive_load`	Avoid unnecessary complexity in question phrasing. Clear, concise language ensures we assess the skill, not reading comprehension.
`constructive_alignment`	Ensure exam questions directly align with the stated learning objectives of the course.
`disclaimer`	This analysis is AI-generated and should serve as an assistive tool, not a definitive authority.

These guidelines are formatted and injected into every prompt, ensuring the LLM's recommendations are *grounded in established pedagogy* rather than generating unsupported claims.

6.3.3 Component 3: Data Validation and Noisy Data Handling

Before generating a report, the agent performs four data quality checks on the input DataFrame:

1. **Missing titles:** Counts questions where the `Title` column is `NaN`.
2. **Missing bodies:** Counts questions where `question_body` is `NaN`.
3. **Zero-score anomaly:** If more than 30% of questions have a difficulty score of exactly 0, this suggests sparse answer data and is flagged.
4. **Small sample warning:** If the dataset contains fewer than 100 questions, the agent warns that analysis reliability may be limited.

These quality notes are appended to the data summary and explicitly included in the LLM prompt with the instruction to "handle gracefully." This ensures the generated report acknowledges data limitations rather than presenting potentially misleading conclusions as fact.

6.4 Structured Report Generation

The LLM is prompted to generate a report with exactly six sections, matching the project requirements:

1. **Assessment Quality Summary:** Overall evaluation of exam balance.
2. **Question Difficulty Distribution:** Statistical summary of Easy/Medium/Hard counts.
3. **Identified Learning Gaps:** Topics where students appear to struggle, inferred from the hardest questions.
4. **Recommended Assessment Improvements:** Actionable advice for educators.
5. **Supporting Pedagogical References:** Citations of the RAG-injected guidelines.
6. **Educational and Ethical Disclaimers:** Explicit statement that the analysis is AI-assisted.

6.5 Prompt Engineering Strategies

To reduce unsupported educational claims, the following prompt engineering strategies are employed:

- **Role assignment:** The LLM is assigned the role of "expert Educational Assessment Designer and AI Agent."
- **Grounding instruction:** "Avoid making unsupported educational claims outside of the provided guidelines."
- **Context injection:** All five pedagogical guidelines are included verbatim in every prompt.
- **Structured output format:** The prompt specifies the exact six-section format, preventing the model from generating unstructured or tangential content.

6.6 Extension: Automated Question Generation

As our chosen extension feature, the system can generate new exam questions on demand. Given a topic name (e.g., "Dynamic Programming") and a difficulty level (Easy/Medium/Hard), the agent prompts Gemini to produce:

1. A question title.
2. A detailed problem description with constraints.
3. Sample input/output examples.
4. A pedagogical justification explaining why the question effectively tests the concept.

This feature closes the feedback loop: the Assessment Report identifies weak topics → the educator uses Automated Question Generation to create targeted questions → future exams are better balanced.

7 System Implementation

7.1 Technology Stack

Table 7: Technology stack used in the ExamWise system

Component	Technology	Purpose
Programming Language	Python 3.x	Core development language
Data Processing	Pandas, NumPy	DataFrame operations, numerical computation
Machine Learning	Scikit-learn	Vectorisation, classification, evaluation
Visualisation	Matplotlib	Charts and plots
Web Interface	Streamlit	Interactive front-end
Model Serialisation	Joblib	Save/load trained models
LLM Integration	Google Generative AI	Gemini API access
Configuration	python-dotenv	Environment variable management

7.2 Project Structure

```
ExamWise/
  app.py                          # Streamlit application (205 lines)
  requirements.txt                # Python dependencies (8 packages)
  .env.example                    # API key template
  .gitignore                      # Excludes data/, venv/, .env
  README.md                       # Project documentation

  scripts/
    data_reduction.py             # Dataset reduction (50 lines)
    feature_engineering.py        # Feature computation (79 lines)
    assessment_agent.py           # Agentic AI assistant (128 lines)

  models/
    model.joblib                  # Trained LR classifier (758 KB)
    vectorizer.joblib             # Fitted CountVectorizer (447 KB)
    metrics.txt                   # Classification report

  data/                           # (not version-controlled)
    raw/                          # User-uploaded CSVs
    reduced/                      # Post-reduction CSVs
    processed/                    # Final labelled dataset

  notebooks/
    model_training.ipynb          # EDA and model training
```

```

  outputs/                          # Generated visualisations
  docs/                             # Project documentation
  report/                           # This LaTeX report
```

Listing 2: Complete repository file structure

7.3 User Interface Design

The Streamlit application (`app.py`) provides a six-section interactive interface:

Table 8: Streamlit UI sections and their functionality

#	Section Title	Functionality
1	Upload Dataset Files	Three file uploaders for Questions, Answers, Tags CSVs
2	Run Full Analysis	One-click button executes the entire M1 pipeline
3	Predicted Difficulty Levels	Data table comparing true vs. predicted labels
4	Model Performance	Classification report + visual confusion matrix heatmap
4.5	Student Performance Patterns	Per-question accuracy analysis table
5	Analytics and Visualisations	Difficulty distribution charts, accuracy histograms
6	Agentic Assessment Assistant	Report generation button + automated question generator

The sidebar provides a secure text input for the Gemini API key (type="password"), and the key can also be loaded from a `.env` file for convenience.

7.4 Security Considerations

- The `.env` file containing the API key is excluded from version control via `.gitignore`.
- A `.env.example` template is provided so new developers know which variables to configure.
- The Streamlit sidebar uses `type="password"` to mask the API key input.
- Raw data files (`data/`) are not committed to the repository.

8 Results: Milestone 2

8.1 Sample Assessment Report

When the "Generate Assessment Design Report" button is clicked, the agent produces a structured markdown report. Below is a representative excerpt from a test run:

> *"The current difficulty distribution (75.2% Hard, 14.6% Medium, 10.2% Easy) deviates significantly from the recommended 20-60-20 ratio. This imbalance suggests that the exam may be overly challenging, potentially discouraging students and failing to properly assess foundational knowledge."*

The agent correctly identified the imbalance using the injected pedagogical guideline and provided actionable recommendations, including increasing the proportion of easy questions and revising question phrasing to reduce extraneous cognitive load.

8.2 Sample Generated Question

The Automated Question Generation feature was tested with the topic "Dynamic Programming" at difficulty level "Hard." The agent produced a complete question including a problem description, constraints, sample input/output, and a pedagogical justification explaining why the problem effectively tests understanding of optimal substructure and memoisation.

8.3 Noisy Data Handling Demonstration

When tested with a small dataset containing missing titles and body text, the agent:

1. Detected and reported the number of missing titles and bodies.
2. Flagged the small sample size as a reliability concern.
3. Generated a report that explicitly acknowledged these limitations.
4. Did not crash or produce misleading output.

9 Discussion

9.1 Strengths

1. **End-to-end integration:** The system seamlessly connects data ingestion, ML prediction, AI-powered analysis, and interactive UI in a single application.

2. **Modular architecture:** Each pipeline stage (reduction, engineering, prediction, analysis) is encapsulated in a separate module, enabling independent testing and future improvements.

3. **Pedagogical grounding:** The synthetic RAG approach ensures that AI recommendations are anchored in established educational principles, not hallucinated claims.

4. **Graceful degradation:** The system handles missing data, empty datasets, and absent API keys without crashing, providing informative warnings instead.

5. **Free-tier compliance:** The entire system runs on free-tier services (Gemini Flash, Streamlit Cloud), making it accessible and cost-effective.

9.2 Limitations

1. **Class imbalance (Milestone 1):** The 75% Hard class dominance causes the classifier to be heavily biased. Techniques such as SMOTE oversampling, class weight adjustment (`class_weight='balanced'`), or stratified sampling should be explored.

2. **Single model baseline:** Only Logistic Regression was trained. Comparing with Decision Trees, Random Forests, SVMs, or gradient-boosted models would provide more robust baselines and potentially better minority-class performance.

3. **Simple vectorisation:** CountVectorizer captures word presence but ignores word importance. TF-IDF weighting or contextual embeddings (BERT) would improve discriminative power.

4. **Proxy difficulty labels:** Stack Overflow engagement metrics are an imperfect proxy for actual exam difficulty. Real student response data would provide more accurate ground-truth labels.

5. **LLM dependency:** The Milestone 2 agent requires an active internet connection and a valid API key. Offline operation is not supported.

6. **Synthetic vs. full RAG:** The hardcoded knowledge base is limited to five guidelines. A full vector-store RAG system with a larger pedagogical corpus would improve the depth and specificity of recommendations.

10 Future Work

1. **Advanced NLP models:** Replace CountVectorizer with TF-IDF or pre-trained transformer models (e.g., BERT, DistilBERT) for richer text representations that capture semantic meaning.

2. **Class imbalance handling:** Apply SMOTE, ADASYN, or adjust `class_weight` in scikit-learn to improve minority class recall.

3. **Multi-model comparison:** Train and evaluate Decision Trees, Random Forests, SVM, and XGBoost classifiers to identify the best-performing model.

4. **Full RAG pipeline:** Integrate a vector store (Chroma or FAISS) with a comprehensive pedagogical knowledge base sourced from education research papers.

5. **LangGraph agent:** Migrate the agentic workflow to LangGraph for more sophisticated multi-step reasoning with tool use capabilities.

6. **Real exam data:** Partner with educational institutions to collect actual student response data for ground-truth difficulty calibration.

7. **PDF export:** Implement PDF generation of the assessment report for formal documentation.

8. **Multi-exam comparative analysis:** Enable side-by-side comparison of multiple exam datasets to track assessment quality trends over time.

11 Conclusion

This project demonstrates that combining classical machine learning with modern generative AI creates a powerful, practical tool for educational assessment design.

Milestone 1 established a complete data pipeline from raw Stack Overflow data to difficulty predictions, achieving 65.71% accuracy with a Logistic Regression baseline. While limited by class imbalance and simple vectorisation, it provides a functional foundation and clearly identifies the challenges inherent in automated difficulty estimation.

Milestone 2 transformed these quantitative predictions into qualitative, actionable insights using a stateful Gemini-powered agent. By injecting pedagogical guidelines via synthetic RAG and implementing robust noisy data handling, the system produces trustworthy, grounded recommendations that respect the boundaries of AI-assisted analysis. The Automated Question Generation extension closes the assessment improvement loop, enabling educators to immediately create targeted content for identified weak areas.

Together, the two milestones deliver a complete assessment lifecycle tool—from retrospective analysis of past exams to proactive design of future ones—accessible through a clean, interactive Streamlit interface and deployable on free-tier cloud platforms.

References

Ashton Anderson, Daniel Huttenlocher, Jon Kleinberg, and Jure Leskovec. Discovering value from community activity on focused question answering sites: A case study of stack overflow. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 850–858. ACM, 2012.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

Le An Ha, Victoria Yaneva, Peter Baldwin, and Janet Mee. Automated estimation of item difficulty for multiple-choice tests: An application of word embedding techniques. In *Proceedings of the 14th Workshop on Innovative Use of NLP for Building Educational Applications*, pages 370–382, 2019.

Ronald K. Hambleton, Hariharan Swaminathan, and H. Jane Rogers. *Fundamentals of Item Response Theory*. Sage Publications, Newbury Park, CA, 1991.

Joblib Development Team. Joblib: Running python functions as pipeline jobs. https://joblib.readthedocs.io/. Accessed: April 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum Associates, Hillsdale, NJ, 1980.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Stack Overflow. StackSample: 10% of Stack Overflow Q&A. Kaggle, 2016. URL https://www.kaggle.com/datasets/stackoverflow/stacksample. Accessed: April 2026.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

A Key Code Listings

A.1 Difficulty Score Computation

```python
# difficulty_score = 1 - (max_answer_score / (max_answer_score + answer_count + 1))
df['difficulty_score'] = 1 - (
    df['max_answer_score'] /
    (df['max_answer_score'] + df['answer_count'] + 1)
)

# Normalize between 0 and 1
ds_min = df['difficulty_score'].min()
ds_max = df['difficulty_score'].max()
if ds_max > ds_min:
    df['difficulty_score'] = (df['difficulty_score'] - ds_min) / (ds_max - ds_min)
else:
    df['difficulty_score'] = 0.5  # Fallback

# Categorize
def categorize(score):
    if score < 0.33: return 'easy'
    elif score < 0.66: return 'medium'
    return 'hard'

df['difficulty_label'] = df['difficulty_score'].apply(categorize)
```

Listing 3: Core difficulty scoring logic from `feature_engineering.py`

A.2 Agent Prompt Construction

```python
prompt = f"""
You are an expert Educational Assessment Designer and AI Agent.
Your task is to analyze the following assessment data and generate
a structured assessment design report.

{kb_context}  # Injected pedagogical guidelines (RAG)

Data Summary:
{data_summary}  # Aggregated metrics + quality flags

Requirements for the report:
1. Assessment Quality Summary
2. Question Difficulty Distribution
3. Identified Learning Gaps
4. Recommended Assessment Improvements
5. Supporting Pedagogical References
6. Educational and Ethical Disclaimers
```

```

Avoid making unsupported educational claims outside of the
provided guidelines.
"""
```

Listing 4: Prompt template for assessment report generation

B Team Contributions

Table 9: Role-wise work allocation across both milestones

Role	Contributions
Person 1 (Data Pipeline)	Dataset acquisition, data reduction script, chunked CSV reading, referential integrity logic, processed CSV export
Person 2 (ML & NLP)	Feature engineering, difficulty scoring formula, CountVectorizer pipeline, Logistic Regression training, model evaluation, confusion matrix
Person 3 (AI Agent)	Gemini API integration, AssessmentAgent class, synthetic RAG knowledge base, prompt engineering, noisy data validation, automated question generation
Person 4 (UI & Integration)	Streamlit application, sidebar configuration, UI layout, visualisation integration, documentation, Git management, deployment